



Code

Archive



adapt-ffb-joy - SidewinderFFBMIDI.wiki

[Export to GitHub](#)

Controlling Force Feedback of Microsoft Sidewinder Force Feedback Pro Joystick

This document is a working document on reverse engineering Microsoft Sidewinder Force Feedback Pro Joystick (FFP) force feedback (FFB) functionality. The long term goal is to be able to develop adapter software also for the joystick's force feedback functionality via a USB-adapter and this is only one step towards that goal. Since Microsoft has already years ago pulled the plug in supporting this game port joystick at any level, I see no shame in reverse engineering its features in order to be able to use the joystick on more modern operating systems and hardware platforms and avoid creating polluting waste out of perfectly functioning piece of hardware.

The content is far from complete and it does contain errors. You have been warned. But, please, help develop this further.

Protocol basics

The joystick's force feedback effects are primarily controlled using game port's MIDI OUT i.e. pin 12. The protocol used seem to follow quite closely MIDI standards for message framing, structure, data types and even in some special values. There are only four types of MIDI-messages being used; message types (0xA, 0xB and 0xC) on channel 6 and SysEx messages.

The first byte of each message has its most significant bit (MSB) set to 1 while all subsequent bytes belonging to the same message has their MSB's set to 0. The first byte contains the command in the first nibble (i.e. 4 bits) and the channel numbers in the second nibble. Subsequent bytes contain the parameters and extra data for the command. As all messages are delivered in MIDI channel 6, the second nibble is always 5 (i.e. 6th channel counting from 0). Subsequent bytes in the message are considered as parameters and their count varies by the command.

Exception to this rule are messages that start with byte value 0xF0 and end with 0xF7 - which corresponds to MIDI standard's SysEx-message type for carrying custom messages. The bytes between these start and stop markers is the actual message data.

Effects data is uploaded to the joystick using the SysEx-messages. Each effect has their own, slightly different data structure. The last data byte of the SysEx-messages is a checksum (XOR of 7E (check this?) like in some MIDI standardized SysEx-messages too) of the message's other data bytes.

0xF0 ...n-bytes data... Checksum 0xF7

The joystick can be uploaded multiple effects. Sending multiple SysEx-messages with effects data, each effect will automatically get assigned an identifier number, which later is referred by commands e.g. to start, stop, modify and

remove the effect. Effect IDs start from number 2. Uploading an effect does not yet start to play it.

An effect is started with 3-byte message

0xB5 0x20 0x02

where the last byte is actually the effect ID to be started and naturally may vary.

For example, uploading an effect (a constant force towards right direction) and immediately playing it could be as (formatted to multiple lines to improve readability):

0xf0 -- start of effect data

*0x00 0x01 0x0a 0x01 0x23 0x12 0x7f 0x5a 0x19 0x00 0x00 0x0e 0x02 0x7f 0x64 0x00 0x10
0x4e 0x7f 0x00 0x00 0x7f 0x5a 0x19 0x7f 0x01 0x00 0x7f 0x00 0x00 0x00 0x18 -- checksum
of data*

0xf7 -- end of effect data 0xb5 0x20 0x02 -- play effect ID 2

The checksum in SysEx-command is calculated from the data after the "0x00 0x01 0x0a 0x01"-part i.e. from the 5th byte in the SysEx-command data. Checksum is calculated by * sum all bytes * divide by 0x80 and take the remainder * subtract the remainder from 0x80 => checksum

Effects

FFP supports quite a wide range of waveforms: * sine * square * ramp * triangle * constant * spring * inertia * friction

(sawtooth is not supported by FFP)

Each effect waveform is associated with a number of waveform-specific parameter sets. Different effect parameters have different data types (that likely have their roots in MIDI-specs). Some types are here: * u14 (unsigned 14-bit)

*Value = Byte1 + Byte2 x 128 (i.e. Byte1 | Byte2 <<7) Max value of each byte is 0x7f * u14i is like u14, but 0x0000=INFINITE * s14 (signed 14-bit) Value = Max value of each byte is 0x7f 0x7f00=+max, 0x0101=-max*

The following parameters are known, but not necessarily all applicable to all waveforms: * Duration [u14i, 2ms] * Direction [u14, deg] * axis0 Offsets and Positive Coefficient * axis1 Offsets and Positive Coefficient * envelope Attack Time * envelope Fade Time * envelope Attack Byte1 used * envelope Fade * periodic Wavelength [Byte1, 0x01..0x6F => 1/Byte1 Hz] * periodic Amplitude * Constant * Ramp Start and Ramp End * (periodic Phase-parameter is not supported by FFP)

Effect data is structured as:

[Byte|Description| |---|:-----| |1..5|Effect header: 0x00 0x01 0x0A 0x01 0x23| |6|Waveform (2=sine, 5=Square, 6=Ramp, 8=Triange, 0x12=Constant, 0xd=Spring, 0xf=Inertia, 0x10=Friction)| |(Waveform dependent parameters - some waveforms even have less bytes than others)| |7| | |8|Duration LSB [u14i, 2ms]| |9|Duration MSB| |10| | |11| |12|Direction B1 [u14, deg]| |13|Direction B2 i.e. 0=0x0000, 90=0x5a00, 180=0x3401, 270=0x0e02| |14| | |15| |16| |17| |18| |19|Envelope Attack| |20|Envelope Attack Time| |21|Envelope Attack Time| |22|Envelope Fade Time| |23|Envelope Fade Time| |24| |25|Envelope Fade| |26|Wavelength [0x6F..0x01 => 1/value Hz]| |27| |28|Constant Dir?| |29|Constant Dir?| |30| |31| |32|Checksum (XOR of 0x7E?)|

Controlling effects

Effects can be started, stopped, removed and altered using the 0xB5 command (MIDI "Control Change"). The message always has 2 parameter bytes with the following meaning. The second byte is the effect ID number and the first byte says what to do to it. If the effect ID is 0x7E, it refers to ALL effects.

0x10 = Remove the effect (also stops the effect if currently playing) 0x20 = Start playing the effect 0x30 = Stop the effect 0x40-0x7C = Modify the effect data on-fly)

) The Modify-effect message is always followed by another 0xA5 message (MIDI "After Touch"), which identifies the new value to the effect parameter indicated by the actual Modify-value. The Modify-value (0x40-0x7C could be seen as an offset to the effect data, but it does not seem to be quite 1-to-1 with it).

For example, message pair

0xB5 0x40 0x02 0xA5 0x01 0x30

will change duration-parameter of the effect ID 2 to value 280 milliseconds (= 0x01*256ms + 0x18*2ms)

and message pair

0xB5 0x48 0x02 0xA5 0x5a 0x00

will change the direction-parameter of the effect ID 2 to value "90 degrees".

Here is a partial list of some parameters Modify-value:

[Offset in original effect data|Modify-value|Parameter and notes] | :-----| :-----| :-----| | 8
|0x40 |Duration | |10 |0x48 |Direction, Spring: Axis0, Positive Coeff| |12 |0x4c |Spring: Axis1, Positive Coeff| |14 |0x50
|Spring: Axis0, Offset| |16 |0x54 |Spring: Axis1, Offset| |18 | |20 |0x5c |Envelope Attack Time| |22 |0x60 |Envelope
Fade Time | |24 |0x64 |Envelope Attack (only byte1 used)| |26 | |28 |0x6c |Envelope Fade | |30 |0x70 |Periodic
Wavelength (byte1 0x6f..0x01) - 1/Hz| |32 |0x74 |Constant, Ramp 1, Periodic: Offset/Amplitude (?)| |34 |0x78 |Ramp
2 (>= Ramp1), Periodic: Offset/Amplitude (?)|

Switching modes

The joystick is by default in a mode where it does not process any FFB messages from the MIDI line.

Once the joystick is in the FFB-mode (i.e. it processes the FFB messages from the MIDI-line), it can still be set into other modes where it behaves differently. For example, by default, the joystick runs an "auto-center-spring-effect". Typically, when an application that produces FFB effects comes to foreground, the auto-center-spring is turned off and when the application quits or goes to background, the auto-center-spring-effect is enabled and all other effects are stopped and removed.

Here's how applications typically enables the joystick when starting up (disables the auto-center-spring effect):

Enable FFB

The below procedure enables the FFB mode in the joystick. It also disables the auto-center spring effect.

It is sufficient to send pulses to X1-line. A single pulse has a form of 50us high and 150us low. A single "command" is composed of one or more pulses in series. Between commands, there is usually a pause of several milliseconds.

1 pulse (wait 7 ms) 4 pulses (wait 24-41 ms) 3 pulses (wait 15 ms) 2 pulses (wait 78 ms) 2 pulses (wait 4 ms) 3 pulses (wait 59 ms) 2 pulses

I.e. omitting the timings, the command sequence would be "1-4-3-2-2-3-2".

The first MIDI bytes "0xC5 0x01..." can be sent immediately after all of the above sequence pulse commands have been completed. The full MIDI data is as follows:

```
0xc5, 0x01 // <ProgramChange> 0x01 (wait for 20 ms) 0xf0, 0x00, 0x01, 0x0a, 0x01, 0x10, 0x05, 0x6b, 0xf7
(wait for 56 ms) 0xb5, 0x40, 0x7f, // <ControlChange>(Modify, 0x7f) 0xa5, 0x72, 0x57, // offset 0x72 :=
0x57 0xb5, 0x44, 0x7f, // ... 0xa5, 0x3c, 0x43, 0xb5, 0x48, 0x7f, 0xa5, 0x7e, 0x00, 0xb5, 0x4c, 0x7f,
0xa5, 0x04, 0x00, 0xb5, 0x50, 0x7f, 0xa5, 0x02, 0x00, 0xb5, 0x54, 0x7f, 0xa5, 0x02, 0x00, 0xb5, 0x58,
0x7f, 0xa5, 0x00, 0x7e, 0xb5, 0x5c, 0x7f, 0xa5, 0x3c, 0x00, 0xb5, 0x60, 0x7f, 0xa5, 0x14, 0x65, 0xb5,
0x64, 0x7f, 0xa5, 0x7e, 0x6b, 0xb5, 0x68, 0x7f, 0xa5, 0x36, 0x00, 0xb5, 0x6c, 0x7f, 0xa5, 0x28, 0x00,
0xb5, 0x70, 0x7f, 0xa5, 0x66, 0x4c, 0xb5, 0x74, 0x7f, 0xa5, 0x7e, 0x01, 0xc5, 0x01 // <ProgramChange> 0x01
(wait for 69 ms) 0xb5, 0x7c, 0x7f, 0xa5, 0x7f, 0x00, 0xc5, 0x06 // <ProgramChange> 0x06
```

The joystick also sends an acknowledgement of the mode switch to the PC. If there is a problem in a sequence, Windows will notify that it cannot find a force feedback device and the above commands are not carried thru, but end only as e.g. sequence "1-4-3-2". The acknowledgement is signaled using the button-lines.

Switch Away from FFB Application

Switch away from application (effects are stopped):

0xC5 0x06

Switch Back to FFB Application

Switch (back) to application:

0xC5 0x01 (wait 69-72 ms) 0xB5 0x7C 0x7F 0xA5 0x7F 0x00 0xC5 0x06

Quit FFB Application

Quit application (enables auto-center spring and puts the joystick off the FFB-mode!):

*0xC5 0x01 (wait 20 ms) 0xC5 0x07 0xB0 0x40 0x00 0xB1 0x40 0x00 0xB2 0x40 0x00 0xB3 0x40 0x00 ...
(continuing for all 16 channels - and twice for all)*

Some sample effects

===RAMP=== (Y-axis, starting from up and sliding downwards) 6.580645 sec.:

00 01 0a 01 23 06 7f 74 17 00 00 00 00 7f 64 00 10 4e 7f 00 00 7f 74 17 7f 01 00 7f 00 01 01 02

===CONSTANT=== (downwards i.e. direction 0) 6.580645 sec.:

00 01 0a 01 23 12 7f 74 17 00 00 00 00 7f 64 00 10 4e 7f 00 00 7f 74 17 7f 01 00 7f 00 00 00 78

to left i.e. direction 90-deg:

00 01 0a 01 23 12 7f 5a 19 00 00 5a 00 7f 64 00 10 4e 7f 00 00 7f 5a 19 7f 01 00 7f 00 00 00 4e

direction "dec 25" = 0x19:

00 01 0a 01 23 12 7f 5a 19 00 00 00 00 7f 64 00 10 4e 7f 00 00 7f 5a 19 7f 01 00 7f 00 00 00 28

direction right i.e. 270-deg:

00 01 0a 01 23 12 7f 5a 19 00 00 0e 02 7f 64 00 10 4e 7f 00 00 7f 5a 19 7f 01 00 7f 00 00 00 18

Others

00 01 0a 01 23 06 7f 5a 19 00 00 00 00 7f 64 00 10 4e 7f 00 00 7f 5a 19 7f 01 00 7f 00 01 01 32 (Ramp)

00 01 0a 01 23 12 7f 5a 19 00 00 00 00 7f 64 00 10 4e 7f 00 00 7f 5a 19 7f 01 00 7f 00 00 00 28
(Constant)

00 01 0a 01 23 05 7f 5a 19 00 00 00 00 7f 64 00 10 4e 7f 00 00 7f 5a 19 7f 01 00 7f 00 01 01 33 (1Hz
square)

00 01 0a 01 23 05 7f 5a 19 00 00 2c 00 7f 64 00 10 4e 7f 00 00 7f 5a 19 7f 01 00 7f 00 01 01 07

00 01 0a 01 23 02 7f 09 16 00 00 00 00 7f 64 00 10 4e 7f 00 00 7f 09 16 7f 01 00 7f 00 01 01 5e (1Hz
sine)

00 01 0a 01 23 0d 7f 09 16 00 00 7f 00 7f 00 00 00 00 00 34 (Spring)

00 01 0a 01 23 10 7f 09 16 00 00 7f 00 7f 00 31 (Friction)

00 01 0a 01 23 0f 7f 09 16 00 00 65 00 65 00 00 00 00 66 (Inertia)